Final Project Documentation

Basics of Programming II

Dr. Dmitriy Dunaev

In the software development process, software documentation is the information that describes the product to the people who develop, deploy and use it.

The Final Project does not only include the C++ program itself, but also the documentation of the program. Writing a document gives students practice with producing "manuals" and "technical documentations", such as those required for real life software development projects. It is important that both the functionality and the implementation of a program are well illustrated for other people who might need this information – even a thoroughly commented source code will not be sufficient to describe larger, more complex software. The documentation will be used as the primary source of information, so designing it should not be taken lightly. **The quality of the documentation has a considerable effect on the overall assessment of the Final Project.**

In real life industrial projects, software documentation is a way for engineers and programmers to describe their product and the process they used in creating it in formal writing. Early computer users were sometimes simply given the engineers' or programmers' notes. As software development became more complicated and formalized, technical writers and editors took over the documentation process.

> *A software developer's work is never done. They're always thinking about the next project, and how to make it better than their last one. The problem with that mindset is that it often leads to neglecting documentation, which has several important benefits:*
>
> - *Help future developers maintain your code*
> - *Can help you learn from your mistakes now so you won't repeat them*
>
> *Documentation means more time spent coding and less time writing instructions for other people. It also means fewer bugs because the code will be easier to understand for everyone involved in the process.*
>
> *You may not think of yourself as someone who needs documentation for your C++ Final Project right now, but trust me - having some handy before starting on a real life project can save you loads of time later.*

Title Page

This page includes the course name (**Basics of Programming 2**), course ID (check in Neptun), full name and Neptun code of the student, lab group ID, name of the lab instructor, project title, type of the project (**Final Project**), and submission date.

Introduction

An introduction stating the problem definition and an overview of the solution detailed in the following sections of the document is given. It may also include some necessary terminology and definitions that will be used throughout the document; in the case that there are plenty of terminology and definitions, they may be written in a separate section.

Program Interface

This section explains how the user communicates with the program. It must be stated how the user will start running the program, i.e. how the programming environment will be set and activated, what is the command to be entered to execute the program, and what are the parameters (if any) that must accompany the execution command. It must also be stated how the user terminates the program. This section is just for activating and deactivating the programming environment; the details of the communication steps will be explained in the following sections.

Program Execution

This section explains in detail the execution of the program from an end-user's point of view. The descriptions of the algorithms, data structures, implementation, etc. should not be included. This section covers what types of inputs will be supplied to the program by the user, what are the outputs of the program, what is the functionality of the program, the descriptions of menu options and different parts of the program, etc. This section can be thought as the core of the "user manual" of the program. It is good practice to include screen outputs and some figures (e.g. expressing the structure and parts of the system). Do not give all the explanations in a single section; instead, divide this section into subsections for ease of reading.

Input and Output

The format of the input and output will be explained in this section. If the program also gets some input from input files and produces some output on output files, then the exact format of the files must also be given. It is advisable to include some example inputs and outputs, and their explanations.

Program Structure

This is the "technical manual" of the program. An overall structure of the program should be given. Then, each part of the program (subprograms, modules, classes, etc.) should be explained in sufficient detail such that the reader can understand the statements, algorithmic logic, etc. The data structures (data types, variables, containers, etc.) should be explained. If desired, the explanation of the data structures can be covered in a separate section (this may be better if the entire program makes use of similar data structures) and this section is reserved for detailing the program code. Do not give all the explanations in a single section; instead, divide this section into subsections for ease of reading.

Program Interface

This section explains how the user communicates with the program. It must be stated how the user will start running the program, i.e. how the programming environment will be set and activated, what is the command to be entered to execute the program, and what are the parameters (if any) that must accompany the execution command. It must also be stated how the user terminates the program. This section is just for activating and deactivating the programming environment; the details of the communication steps will be explained in the following sections.

Examples (optional)

It will facilitate understanding the program and the document if some examples regarding the execution are included. For instance, a particular input can be given and the operations and the output of the program under this input may be specified. The examples should be chosen in a manner such that different behaviours of the program may be observed under different inputs.

Testing and Verification

Testing and verification are essential aspects of any software product. Without testing, you cannot be sure of what you are giving the customer or the user. At any time, an unexpected problem may arise that is not obvious when looking at the code. How did you make sure that your program does not contain errors (testing)? How did you make sure that it does what it should (verification)? Describes the process, objectives, and results of software testing. You can also include information on the environment, setup, and configuration required to perform testing. The goal of this chapter is to communicate the details of a test plan or strategy.

Improvements and Extensions

In this section, the parts of the program that need improvement and some possible future extensions are discussed. The weak and strong points of the program can be identified. Any shortcomings or defects of the program can be stated. Also, the deviations from the Application Form, which you submitted at the beginning of the project can be indicated, i.e. the features, user options, data structures, etc. that were initially planned to be included, but could not be done so for some reasons.

Difficulties Encountered

It is quite valuable for an educational project to identify clearly the difficulties encountered within the project period. The difficulties due to the programming language (maybe it is a new language for you), due to the nature of the problem, etc. should be stated. Also, the effects of these difficulties (forcing to use different ways, abandoning some planned features of the program, etc.) should be explained.

Conclusion

This section is for the conclusions about the project.

References

Each time you quote or write a passage inspired by something you might have read elsewhere, you must include a source reference – this applies to books, magazines, websites, etc. Every time you base your writing (coding) on information found elsewhere, you must include a source reference. This also applies to your own previous work, such as something you might

have written yourself in previous projects. It is highly essential that your documentation includes accurate source references. Taking notes as you go along will make it easier for you to compile your final reference list, and you minimise the risk of finding yourselves in a situation in which you cannot find a particular reference. You may also use RefWorks or Mendeley which are tools used for collecting and managing references. Microsoft Word also allows you to create a 'bibliography'. For the Final Project documentation, we advise you using the IEEE citation and reference format.

Appendicies (optional)

Large (10+ lines) source code snippets, which are referenced in the documentation, must be included as an appendix. Also, if necessary, some information can be given inside appendices. For instance, an appendix is a suitable place for the execution details for some complex sample executions.